

Fouzia Zilani

May 2025

1 Compare performance of 10 CNNs pre-trained on ImageNet dataset using images of 20 classes of CIFAR-100 dataset.

We have used 10 CNNs pre-trained models for this problem those are:

1. Xception
2. ResNet50
3. MobileNet
4. MobileNetV2
5. DenseNet121
6. NASNetMobile
7. EfficientNetB0
8. EfficientNetB1
9. EfficientNetB2
10. EfficientNetB3

1.1 Comparison of models

Comparison of the models we have used are given below:

Here, MobileNet and MobileNetV2 were very fast. This models size was the smallest than other models.

EfficientNetB0 and NASNetMobile were comapatively fast. These are smaller than other models expect MobileNet and MobileNetV2 in size.

EfficientNetB1, EfficientNetB2 and DenseNet121 were moderate in time. These models size are relatively larger in size.

EfficientNetB3 and Xception models are slower than other models. These models are the largest in size.

We got the highest accuracy in DenseNet121 and the lowest accuracy in Xception.

Model	Accuracy (%)	Size of the model	Inference time
Xception	72.7	88	Slower
ResNet50	85.9	98	Fast
MobileNet	85.3	16	Very Fast
MobileNet V2	83.2	14	Very Fast
DenseNet121	86.6	33	Moderate
NASNetMobile	79.6	23	Fast
EfficientNetB0	86.2	29	Fast
EfficientNetB1	86.4	31	Moderate
EfficientNetB2	86.5	36	Moderate
EfficientNetB3	85.9	48	Slower

Table 1: Comparison of the models.

1.2 Code for Xception:

```
import tensorflow as tf
from tensorflow.keras.applications import Xception
from tensorflow.keras.applications.xception import preprocess_input
from tensorflow.keras import layers, models
import numpy as np
from tensorflow.keras.datasets import cifar100

(x_train, y_train), (x_test, y_test) = cifar100.load_data(label_mode='fine')
selected_classes = list(range(20))
train_mask = np.isin(y_train, selected_classes).flatten()
test_mask = np.isin(y_test, selected_classes).flatten()

x_train_20 = x_train[train_mask]
y_train_20 = y_train[train_mask]
x_test_20 = x_test[test_mask]
y_test_20 = y_test[test_mask]

label_map = {cls: i for i, cls in enumerate(selected_classes)}
y_train_20 = np.array([label_map[y[0]] for y in y_train_20])
y_test_20 = np.array([label_map[y[0]] for y in y_test_20])

x_train_20 = tf.image.resize(x_train_20, (71, 71)).numpy()
x_test_20 = tf.image.resize(x_test_20, (71, 71)).numpy()

x_train_20 = preprocess_input(x_train_20)
x_test_20 = preprocess_input(x_test_20)

y_train_20_cat = tf.keras.utils.to_categorical(y_train_20, 20)
y_test_20_cat = tf.keras.utils.to_categorical(y_test_20, 20)
```

```

base_model = Xception(weights='imagenet', include_top=False, input_shape=(71, 71, 3))
base_model.trainable = False

model = models.Sequential([
    base_model,
    layers.GlobalAveragePooling2D(),
    layers.Dense(256, activation='relu'),
    layers.Dense(20, activation='softmax')
])

model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])

model.fit(x_train_20, y_train_20_cat, epochs=5, batch_size=64, validation_split=0.1)

loss, acc = model.evaluate(x_test_20, y_test_20_cat)
print("Test accuracy on 20 CIFAR-100 classes using Xception:", acc)

```

1.3 Code for ResNet50:

```

import tensorflow as tf
from tensorflow.keras.applications import ResNet50
from tensorflow.keras.applications.resnet import preprocess_input
from tensorflow.keras import layers, models
import numpy as np
from tensorflow.keras.datasets import cifar100

(x_train, y_train), (x_test, y_test) = cifar100.load_data(label_mode='fine')

selected_classes = list(range(20))
train_mask = np.isin(y_train, selected_classes).flatten()
test_mask = np.isin(y_test, selected_classes).flatten()

x_train_20 = x_train[train_mask]
y_train_20 = y_train[train_mask]
x_test_20 = x_test[test_mask]
y_test_20 = y_test[test_mask]

label_map = {cls: i for i, cls in enumerate(selected_classes)}
y_train_20 = np.array([label_map[y[0]] for y in y_train_20])
y_test_20 = np.array([label_map[y[0]] for y in y_test_20])

x_train_20 = tf.image.resize(x_train_20, (128, 128)).numpy()
x_test_20 = tf.image.resize(x_test_20, (128, 128)).numpy()

x_train_20 = preprocess_input(x_train_20)

```

```

x_test_20 = preprocess_input(x_test_20)

_train_20_cat = tf.keras.utils.to_categorical(y_train_20, 20)
y_test_20_cat = tf.keras.utils.to_categorical(y_test_20, 20)

base_model = ResNet50(weights='imagenet', include_top=False, input_shape=(128, 128, 3))
base_model.trainable = False

model = models.Sequential([
    base_model,
    layers.GlobalAveragePooling2D(),
    layers.Dense(256, activation='relu'),
    layers.Dropout(0.4),
    layers.Dense(20, activation='softmax')
])

model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])

model.fit(x_train_20, y_train_20_cat, epochs=5, batch_size=64, validation_split=0.1)

loss, acc = model.evaluate(x_test_20, y_test_20_cat)
print("Test accuracy on 20 CIFAR-100 classes using ResNet50:", acc)

```

1.4 Code for MobileNet:

```

import tensorflow as tf
from tensorflow.keras.applications import MobileNet
from tensorflow.keras.applications.mobilenet import preprocess_input
from tensorflow.keras import layers, models
import numpy as np
from tensorflow.keras.datasets import cifar100

(x_train, y_train), (x_test, y_test) = cifar100.load_data(label_mode='fine')

selected_classes = list(range(20))
train_mask = np.isin(y_train, selected_classes).flatten()
test_mask = np.isin(y_test, selected_classes).flatten()

x_train_20 = x_train[train_mask]
y_train_20 = y_train[train_mask]
x_test_20 = x_test[test_mask]
y_test_20 = y_test[test_mask]

label_map = {cls: i for i, cls in enumerate(selected_classes)}
y_train_20 = np.array([label_map[y[0]] for y in y_train_20])

```

```

y_test_20 = np.array([label_map[y[0]] for y in y_test_20])

x_train_20 = tf.image.resize(x_train_20, (128, 128)).numpy()
x_test_20 = tf.image.resize(x_test_20, (128, 128)).numpy()

x_train_20 = preprocess_input(x_train_20)
x_test_20 = preprocess_input(x_test_20)

y_train_20_cat = tf.keras.utils.to_categorical(y_train_20, 20)
y_test_20_cat = tf.keras.utils.to_categorical(y_test_20, 20)

base_model = MobileNet(weights='imagenet', include_top=False, input_shape=(128,
base_model.trainable = False # freeze base for transfer learning

model = models.Sequential([
    base_model,
    layers.GlobalAveragePooling2D(),
    layers.Dense(128, activation='relu'),
    layers.Dropout(0.3),
    layers.Dense(20, activation='softmax')
])

model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])

model.fit(x_train_20, y_train_20_cat, epochs=5, batch_size=64, validation_split=0.1)

loss, acc = model.evaluate(x_test_20, y_test_20_cat)
print("Test accuracy on 20 CIFAR-100 classes using MobileNet:", acc)

```

1.5 Code for MobileNetV2:

```

import tensorflow as tf
from tensorflow.keras.applications import MobileNetV2
from tensorflow.keras.applications.mobilenet_v2 import preprocess_input
from tensorflow.keras import layers, models
import numpy as np
from tensorflow.keras.datasets import cifar100

(x_train, y_train), (x_test, y_test) = cifar100.load_data(label_mode='fine')

selected_classes = list(range(20))
train_mask = np.isin(y_train, selected_classes).flatten()
test_mask = np.isin(y_test, selected_classes).flatten()

x_train_20 = x_train[train_mask]

```

```

y_train_20 = y_train[train_mask]
x_test_20 = x_test[test_mask]
y_test_20 = y_test[test_mask]

label_map = {cls: i for i, cls in enumerate(selected_classes)}
y_train_20 = np.array([label_map[y[0]] for y in y_train_20])
y_test_20 = np.array([label_map[y[0]] for y in y_test_20])

x_train_20 = tf.image.resize(x_train_20, (128, 128)).numpy()
x_test_20 = tf.image.resize(x_test_20, (128, 128)).numpy()

x_train_20 = preprocess_input(x_train_20)
x_test_20 = preprocess_input(x_test_20)

y_train_20_cat = tf.keras.utils.to_categorical(y_train_20, 20)
y_test_20_cat = tf.keras.utils.to_categorical(y_test_20, 20)

base_model = MobileNetV2(weights='imagenet', include_top=False, input_shape=(128, 128, 3))
base_model.trainable = False # freeze base

model = models.Sequential([
    base_model,
    layers.GlobalAveragePooling2D(),
    layers.Dense(128, activation='relu'),
    layers.Dropout(0.3),
    layers.Dense(20, activation='softmax')
])

model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])

model.fit(x_train_20, y_train_20_cat, epochs=5, batch_size=64, validation_split=0.1)

loss, acc = model.evaluate(x_test_20, y_test_20_cat)
print("Test accuracy on 20 CIFAR-100 classes using MobileNetV2:", acc)

```

1.6 Code for DenseNet121:

```

import tensorflow as tf
from tensorflow.keras.applications import DenseNet121
from tensorflow.keras.applications.densenet import preprocess_input
from tensorflow.keras import layers, models
import numpy as np
from tensorflow.keras.datasets import cifar100

(x_train, y_train), (x_test, y_test) = cifar100.load_data(label_mode='fine')

```

```

selected_classes = list(range(20))
train_mask = np.isin(y_train, selected_classes).flatten()
test_mask = np.isin(y_test, selected_classes).flatten()

x_train_20 = x_train[train_mask]
y_train_20 = y_train[train_mask]
x_test_20 = x_test[test_mask]
y_test_20 = y_test[test_mask]

label_map = {cls: i for i, cls in enumerate(selected_classes)}
y_train_20 = np.array([label_map[y[0]] for y in y_train_20])
y_test_20 = np.array([label_map[y[0]] for y in y_test_20])

x_train_20 = tf.image.resize(x_train_20, (128, 128)).numpy()
x_test_20 = tf.image.resize(x_test_20, (128, 128)).numpy()

x_train_20 = preprocess_input(x_train_20)
x_test_20 = preprocess_input(x_test_20)

y_train_20_cat = tf.keras.utils.to_categorical(y_train_20, 20)
y_test_20_cat = tf.keras.utils.to_categorical(y_test_20, 20)

base_model = DenseNet121(weights='imagenet', include_top=False, input_shape=(128, 128, 3))
base_model.trainable = False # freeze base

model = models.Sequential([
    base_model,
    layers.GlobalAveragePooling2D(),
    layers.Dense(256, activation='relu'),
    layers.Dropout(0.4),
    layers.Dense(20, activation='softmax')
])

model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])

model.fit(x_train_20, y_train_20_cat, epochs=5, batch_size=64, validation_split=0.1)

loss, acc = model.evaluate(x_test_20, y_test_20_cat)
print("Test accuracy on 20 CIFAR-100 classes using DenseNet121:", acc)

```

1.7 Code for NASNetMobile:

```

import tensorflow as tf
from tensorflow.keras.applications import NASNetMobile

```

```

from tensorflow.keras.applications.nasnet import preprocess_input
from tensorflow.keras import layers, models
import numpy as np
from tensorflow.keras.datasets import cifar100

(x_train, y_train), (x_test, y_test) = cifar100.load_data(label_mode='fine')

selected_classes = list(range(20))
train_mask = np.isin(y_train, selected_classes).flatten()
test_mask = np.isin(y_test, selected_classes).flatten()

x_train_20 = x_train[train_mask]
y_train_20 = y_train[train_mask]
x_test_20 = x_test[test_mask]
y_test_20 = y_test[test_mask]

label_map = {cls: i for i, cls in enumerate(selected_classes)}
y_train_20 = np.array([label_map[y[0]] for y in y_train_20])
y_test_20 = np.array([label_map[y[0]] for y in y_test_20])

x_train_20 = tf.image.resize(x_train_20, (128, 128)).numpy()
x_test_20 = tf.image.resize(x_test_20, (128, 128)).numpy()

x_train_20 = preprocess_input(x_train_20)
x_test_20 = preprocess_input(x_test_20)

y_train_20_cat = tf.keras.utils.to_categorical(y_train_20, 20)
y_test_20_cat = tf.keras.utils.to_categorical(y_test_20, 20)

base_model = NASNetMobile(weights='imagenet', include_top=False, input_shape=(128, 128, 3))
base_model.trainable = False # freeze base for transfer learning

model = models.Sequential([
    base_model,
    layers.GlobalAveragePooling2D(),
    layers.Dense(128, activation='relu'),
    layers.Dropout(0.3),
    layers.Dense(20, activation='softmax')
])

model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])

model.fit(x_train_20, y_train_20_cat, epochs=5, batch_size=64, validation_split=0.1)

loss, acc = model.evaluate(x_test_20, y_test_20_cat)
print("Test accuracy on 20 CIFAR-100 classes using NSANetMobile:", acc)

```


1.8 Code for EfficientNetB0:

```
import tensorflow as tf
from tensorflow.keras.applications import EfficientNetB0
from tensorflow.keras.applications.efficientnet import preprocess_input
from tensorflow.keras import layers, models
import numpy as np
from tensorflow.keras.datasets import cifar100

(x_train, y_train), (x_test, y_test) = cifar100.load_data(label_mode='fine')

selected_classes = list(range(20))
train_mask = np.isin(y_train, selected_classes).flatten()
test_mask = np.isin(y_test, selected_classes).flatten()

x_train_20 = x_train[train_mask]
y_train_20 = y_train[train_mask]
x_test_20 = x_test[test_mask]
y_test_20 = y_test[test_mask]

label_map = {cls: i for i, cls in enumerate(selected_classes)}
y_train_20 = np.array([label_map[y[0]] for y in y_train_20])
y_test_20 = np.array([label_map[y[0]] for y in y_test_20])

x_train_20 = tf.image.resize(x_train_20, (128, 128)).numpy()
x_test_20 = tf.image.resize(x_test_20, (128, 128)).numpy()

x_train_20 = preprocess_input(x_train_20)
x_test_20 = preprocess_input(x_test_20)

y_train_20_cat = tf.keras.utils.to_categorical(y_train_20, 20)
y_test_20_cat = tf.keras.utils.to_categorical(y_test_20, 20)

base_model = EfficientNetB0(weights='imagenet', include_top=False, input_shape=(
base_model.trainable = False # freeze base

model = models.Sequential([
    base_model,
    layers.GlobalAveragePooling2D(),
    layers.Dense(256, activation='relu'),
    layers.Dropout(0.4),
    layers.Dense(20, activation='softmax')
])

model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accu
```

```

model.fit(x_train_20, y_train_20_cat, epochs=5, batch_size=64, validation_split=
loss, acc = model.evaluate(x_test_20, y_test_20_cat)
print("Test accuracy on 20 CIFAR-100 classes using EfficientNetB0:", acc)

```

1.9 Code for EfficientNetB1:

```

import tensorflow as tf
from tensorflow.keras.applications import EfficientNetB1
from tensorflow.keras.applications.efficientnet import preprocess_input
from tensorflow.keras import layers, models
import numpy as np
from tensorflow.keras.datasets import cifar100

(x_train, y_train), (x_test, y_test) = cifar100.load_data(label_mode='fine')

selected_classes = list(range(20))
train_mask = np.isin(y_train, selected_classes).flatten()
test_mask = np.isin(y_test, selected_classes).flatten()

x_train_20 = x_train[train_mask]
y_train_20 = y_train[train_mask]
x_test_20 = x_test[test_mask]
y_test_20 = y_test[test_mask]

label_map = {cls: i for i, cls in enumerate(selected_classes)}
y_train_20 = np.array([label_map[y[0]] for y in y_train_20])
y_test_20 = np.array([label_map[y[0]] for y in y_test_20])

x_train_20 = tf.image.resize(x_train_20, (128, 128)).numpy()
x_test_20 = tf.image.resize(x_test_20, (128, 128)).numpy()

x_train_20 = preprocess_input(x_train_20)
x_test_20 = preprocess_input(x_test_20)

y_train_20_cat = tf.keras.utils.to_categorical(y_train_20, 20)
y_test_20_cat = tf.keras.utils.to_categorical(y_test_20, 20)

base_model = EfficientNetB1(weights='imagenet', include_top=False, input_shape=(
base_model.trainable = False # freeze base

model = models.Sequential([
    base_model,
    layers.GlobalAveragePooling2D(),
    layers.Dense(256, activation='relu'),

```

```

        layers.Dropout(0.4),
        layers.Dense(20, activation='softmax')
    ])

model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])

model.fit(x_train_20, y_train_20_cat, epochs=5, batch_size=64, validation_split=0.1)

loss, acc = model.evaluate(x_test_20, y_test_20_cat)
print("Test accuracy on 20 CIFAR-100 classes using EfficientNetB1:", acc)

```

1.10 Code for EfficientNetB2:

```

import tensorflow as tf
from tensorflow.keras.applications import EfficientNetB2
from tensorflow.keras.applications.efficientnet import preprocess_input
from tensorflow.keras import layers, models
import numpy as np
from tensorflow.keras.datasets import cifar100

(x_train, y_train), (x_test, y_test) = cifar100.load_data(label_mode='fine')

selected_classes = list(range(20))
train_mask = np.isin(y_train, selected_classes).flatten()
test_mask = np.isin(y_test, selected_classes).flatten()

x_train_20 = x_train[train_mask]
y_train_20 = y_train[train_mask]
x_test_20 = x_test[test_mask]
y_test_20 = y_test[test_mask]

label_map = {cls: i for i, cls in enumerate(selected_classes)}
y_train_20 = np.array([label_map[y[0]] for y in y_train_20])
y_test_20 = np.array([label_map[y[0]] for y in y_test_20])

x_train_20 = tf.image.resize(x_train_20, (128, 128)).numpy()
x_test_20 = tf.image.resize(x_test_20, (128, 128)).numpy()

x_train_20 = preprocess_input(x_train_20)
x_test_20 = preprocess_input(x_test_20)

y_train_20_cat = tf.keras.utils.to_categorical(y_train_20, 20)
y_test_20_cat = tf.keras.utils.to_categorical(y_test_20, 20)

base_model = EfficientNetB2(weights='imagenet', include_top=False, input_shape=(128, 128, 3))

```

```

base_model.trainable = False # freeze base

model = models.Sequential([
    base_model,
    layers.GlobalAveragePooling2D(),
    layers.Dense(256, activation='relu'),
    layers.Dropout(0.4),
    layers.Dense(20, activation='softmax')
])

model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])

model.fit(x_train_20, y_train_20_cat, epochs=5, batch_size=64, validation_split=0.1)

loss, acc = model.evaluate(x_test_20, y_test_20_cat)
print("Test accuracy on 20 CIFAR-100 classes using EfficientNetB2:", acc)

```

1.11 Code for EfficientNetB3:

```

import tensorflow as tf
from tensorflow.keras.applications import EfficientNetB3
from tensorflow.keras.applications.efficientnet import preprocess_input
from tensorflow.keras import layers, models
import numpy as np
from tensorflow.keras.datasets import cifar100

(x_train, y_train), (x_test, y_test) = cifar100.load_data(label_mode='fine')

selected_classes = list(range(20))
train_mask = np.isin(y_train, selected_classes).flatten()
test_mask = np.isin(y_test, selected_classes).flatten()

x_train_20 = x_train[train_mask]
y_train_20 = y_train[train_mask]
x_test_20 = x_test[test_mask]
y_test_20 = y_test[test_mask]

label_map = {cls: i for i, cls in enumerate(selected_classes)}
y_train_20 = np.array([label_map[y[0]] for y in y_train_20])
y_test_20 = np.array([label_map[y[0]] for y in y_test_20])

x_train_20 = tf.image.resize(x_train_20, (128, 128)).numpy()
x_test_20 = tf.image.resize(x_test_20, (128, 128)).numpy()

x_train_20 = preprocess_input(x_train_20)

```

```

x_test_20 = preprocess_input(x_test_20)

y_train_20_cat = tf.keras.utils.to_categorical(y_train_20, 20)
y_test_20_cat = tf.keras.utils.to_categorical(y_test_20, 20)

base_model = EfficientNetB3(weights='imagenet', include_top=False, input_shape=(
base_model.trainable = False # Freeze base model

# Build model
model = models.Sequential([
    base_model,
    layers.GlobalAveragePooling2D(),
    layers.Dense(256, activation='relu'),
    layers.Dropout(0.4),
    layers.Dense(20, activation='softmax')
])

# Compile model
model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])

# Train model
model.fit(x_train_20, y_train_20_cat, epochs= 5 , batch_size=64, validation_split=0.1)

# Evaluate model
loss, acc = model.evaluate(x_test_20, y_test_20_cat)
print("Test accuracy on 20 CIFAR-100 classes using EfficientNetB3:", acc)

```